Diagrama de Classe

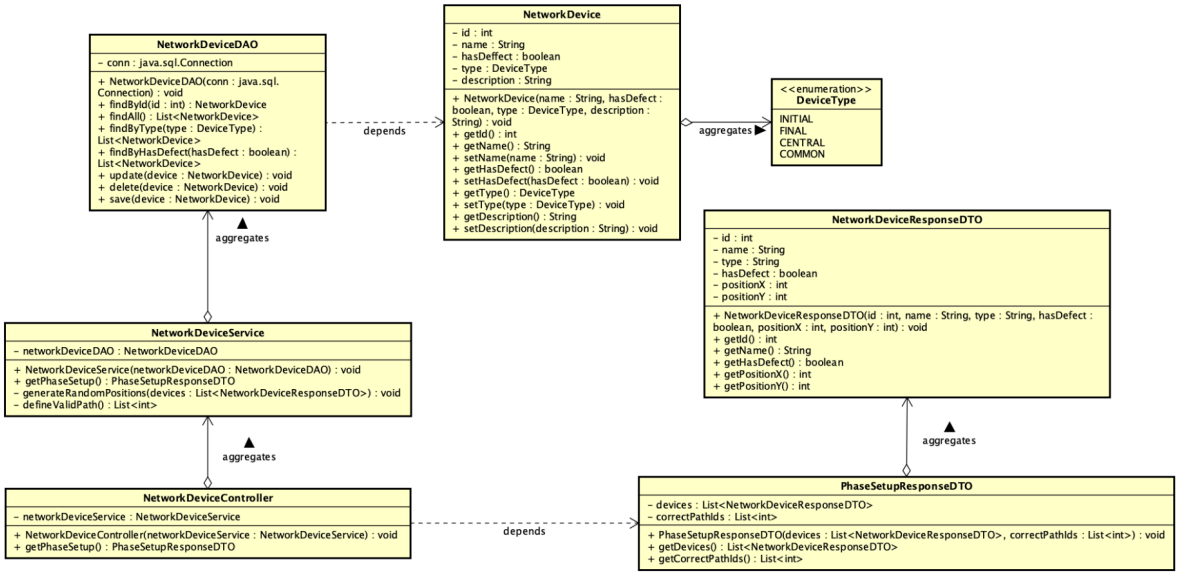
CSU05: Definir caminho da mensagem

Autor: Ana Carolina Fernandes (Arquiteta de Software)

Descrição do caso de uso:

O caso de uso apresentado permite ao sistema gerenciar a fase onde o aluno deve definir o caminho correto para uma mensagem trafegar. O sistema deve buscar os dispositivos de rede no banco de dados, calcular posições aleatórias para os itens, definir um caminho válido entre os pontos inicial e final, e retornar todas essas informações para a interface do jogo.

Diagrama:



Descrição do diagrama: Seguindo a arquitetura em camadas do projeto, o sistema de pontuação é dividido em componentes:

- **Entidade principal:**
    - A classe `NetworkDevice` representa a entidade do dispositivo de rede, contendo informações como identificador (`id`), nome (`name`), se possui defeito (`hasDefect`), o seu tipo (`type`) e uma descrição (`description`).
    - O Enum `DeviceType` define os tipos possíveis para um dispositivo (`INITIAL, FINAL, CENTRAL, COMMON`), garantindo a consistência dos dados.
- **Persistência:**

    - A classe `NetworkDeviceDAO` é a única responsável por todas as operações de banco de dados relacionadas aos dispositivos de rede (buscar todos, buscar por tipo, etc.).
- **Serviços:**
    - A classe `NetworkDeviceService` encapsula toda a lógica de negócio referente à montagem da fase. Ela utiliza uma instância de `NetworkDeviceDAO` para obter os dados brutos e então processá-los. Suas principais responsabilidades são:
        - Gerar posições X e Y aleatórias para os dispositivos.
        - Calcular e definir o caminho correto que a mensagem deve seguir.
        - Converter as entidades `NetworkDevice` para os DTOs que serão enviados na resposta da API.
- **Controladores:**
    - A classe `NetworkDeviceController` atua como a interface de entrada para as requisições do cliente. Ela expõe o endpoint que o frontend chamará para iniciar a fase. Sua única função é receber a chamada, repassá-la para o `NetworkDeviceService` e retornar a resposta formatada.
- **Objetos de Transferência de Dados (DTOs):**
    - O sistema utiliza um padrão que separa os DTOs de entrada (Request) e saída (Response).
    - A classe `NetworkDeviceResponseDTO` formata os dados de um único dispositivo para a resposta, incluindo sua posição calculada (`positionX`, `positionY`) e convertendo o tipo `Enum` para `String`, desacoplando o frontend do modelo interno do backend.
    - A classe `PhaseSetupResponseDTO` encapsula a resposta completa para o cliente, contendo a lista de todos os `NetworkDeviceResponseDTO` e a lista de IDs que formam o `correctPathIds`.

Implementação: Para implementar o sistema, recomenda-se a seguinte abordagem em camadas:

1. **Modelagem das entidades:** Crie o `Enum` `DeviceType` e a classe `NetworkDevice` para representar os dados centrais.
2. **Camada de persistência:** Implemente a classe `NetworkDeviceDAO` para interagir com o banco de dados e realizar as consultas necessárias.
3. **Regras de negócio:** Crie a `NetworkDeviceService`, que conterá toda a lógica de negócio. Utilize a instância de `NetworkDeviceDAO` para buscar os dados e implemente os métodos para gerar as posições e definir o caminho válido.

4. **Camada de controle:** Construa a `NetworkDeviceController`, que receberá as requisições HTTP, chamará o serviço correspondente e retornará o `PhaseSetupResponseDTO`.
5. **Transferência de dados:** Crie as classes `NetworkDeviceResponseDTO` e `PhaseSetupResponseDTO` e utilize-as na camada de controle para expor apenas as informações necessárias como resposta, evitando expor detalhes internos das entidades.